ConnectEase — App Explanation & Presentation Guide

For presenting the ConnectEase project (Tasks 1, 2 & 3) — complete offline-first network diagnostics and connectivity reporting for Nigerian mobile users.

---

1. What Is ConnectEase?

ConnectEase is a **frontend-first, offline-capable web application** that helps users in Nigeria (and similar mobile-first markets) diagnose, report, and track mobile-network connectivity issues — without needing a backend, an account, or an internet connection to use it.

Everything the user creates is stored locally on their device in **IndexedDB**. An optional backend (Node/Express + PostgreSQL) can be connected later to synchronise records, reports and network-test results, but the app is fully functional without it.

Target audience: Mobile users on MTN, Airtel, Glo, and 9mobile who experience dropped calls, slow browsing, and poor signal, and want a simple local tool to record and measure those problems.

---

2. The Three Tasks (What Was Built)

Task	Feature	What It Does
Task 1	**Dashboard**	Offline-first home screen with live connection status, data-saver info, usage stats, offline library, sync queue preview, and recent activity. Data is clearly marked as **demo/sample** so users know it is not real.
Task 2	**Offline Workspace & Sync Centre**	Create, save, and manage local records (notes, contacts, resources) that stay available without a connection. A sync queue tracks pending changes. Settings let the user control data saving and sync behaviour.
Task 3	**Network Test & Connectivity Reports**	**Network test** — measures real latency, download and upload speed against a user-supplied endpoint. No speeds are simulated; if there is no endpoint, it reports "unavailable" honestly. **Network reports** — voluntary poor-network reports

Task	Feature	What It Does
		with provider (MTN/Airtel/Glo/9mobile/Other), location, issue description, and local timestamp. Stored locally, searchable, filterable, with clearly labelled demo data and a map-ready layout.

---

3. Key Features Explained

3.1 Dashboard (Task 1)

- **Overview:** Greeting, date, connection status (online/offline), data-saver chip, plan details (MTN · 3 GB monthly, data used/remaining).
- **Offline library:** Saved items available without data.
- **Demo sync queue:** Shows pending items; preview/clear actions are local-only.
- **Recent activity:** Feed of latest device activity.
- **Explicit demo notice:** "Dashboard figures and sample records are demo data. Your workspace records are stored only in this browser — no backend, account or sync server is connected."

3.2 Offline Workspace & Sync Centre (Task 2)

- **Create records:** Notes, contacts, or resources saved locally in IndexedDB.
- **Pending changes:** Every edit/create/delete is queued as a pending change until sync.
- **Persist across reloads:** Records survive page refresh because they live in IndexedDB.
- **Sync centre:** Shows connection status, last sync time, pending-change count, and a manual "Sync now" button. Works offline (queues changes) and online (sends to backend when configured).

3.3 Network Test (Task 3)

- **Real measurement only:** The browser runs an actual `fetch`-based test:
  1. **Latency** — lightweight GET to the endpoint.
  2. **Download** — streamed download (~1 MB target) measuring throughput.
  3. **Upload** — 256 KB POST payload measuring throughput.
- **User-supplied endpoint:** The user must enter a URL (e.g. `https://example.com/test-file`). Without one, the app reports **"Unavailable — No**

network test endpoint configured" and does **not** invent speeds.

- **Honest errors:** If the endpoint is unreachable or returns an error, the app shows the real error message rather than faking results.
- **History:** Tests are saved locally with timestamp, endpoint, and availability status.

3.4 Network Reports (Task 3)

- **Voluntary reporting:** Users report poor network experiences (dropped calls, slow browsing, no signal indoors, etc.).
- **Provider selection:** MTN, Airtel, Glo, 9mobile, or Other — each with a colour-coded badge.
- **Location:** Optional area/approximate location (privacy reminder: do not share personal addresses).
- **Issue description:** Free-text issue with 1,000-character limit.
- **Local timestamp:** Recorded automatically with local date/time.
- **Stored locally:** Reports live in IndexedDB; can be created and viewed offline.
- **Search & filter:** Search by provider, location, or issue. Filter by provider.
- **Map-ready layout:** Report locations are stored so a map view can be added later (placeholder shown).
- **Demo labelling:** Sample reports (added only when the user clicks "Add sample reports") are clearly labelled **"Demo data — not a real report"** and are **never synced** to the backend.

3.5 Sync Centre (Task 2 & 3)

- **Anonymous installation:** The device generates an anonymous ID + secret stored locally. No user account, no email, no password.
- **Offline-first sync:** Writes happen locally first. Sync sends queued changes, reports, and test results to the backend when online and configured.
- **Backend optional:** If `VITE_API_URL` is not set, the app stays local-only and the Sync page reports "Sync server not configured."
- **Idempotent:** Retrying a sync is safe; duplicate submissions are detected by the backend.

3.6 Settings

- Data saver on/off.
- Image compression, auto-download, sync-on-Wi-Fi preferences.
- Clear statement that preferences and records are stored locally.

---

4. Why This Matters (Importance)

4.1 Solves Real Problems for Nigerian Mobile Users

- **Dropped calls & slow browsing:** Users can record exactly when and where it happens, with their provider and location.
- **No internet needed to record:** The app works fully offline — critical in areas with poor connectivity.
- **Honest measurements:** Unlike many apps that simulate or inflate speeds, ConnectEase only reports real measured values and clearly marks unavailable results.

4.2 Privacy-First by Design

- No accounts, no login, no personal data required.
- All data stays on the device (IndexedDB) unless the user explicitly connects a backend.
- Sample/demo data is isolated and never uploaded.
- Location is optional and users are warned not to share personal addresses.

4.3 Offline-First Architecture

- IndexedDB is the **source of truth**. The UI reads from local storage, so the app is instant and works without a network.
- Sync is an **optional transport layer**, not a dependency. If the backend is down or unconfigured, the app continues to work normally.

4.4 Transparency About Demo Data

- Every screen that uses sample figures has an explicit banner: "Demo preview. Dashboard figures and sample records are demo data."
- Demo reports are visibly tagged "Demo data — not a real report" and excluded from sync.
- This prevents users from mistaking sample data for real measurements.

4.5 Extensible & Production-Ready

- TypeScript throughout with strict type-checking (`tsc --noEmit` passes).
- Vite build + production preview verified.
- Service worker caches the app shell for offline reload.
- Backend integration fully documented in `integration.md` with real, tested API contracts.
- Mobile responsive: tested at 390px and 320px widths with no horizontal overflow.

---

5. App Architecture

```
Frontend (React + TypeScript + Vite)
├── IndexedDB (connectease-offline, v3)   ← Local source of truth
│   ├── records / changes (workspace)
│   ├── reports                            (network reports)
│   ├── networkTests                       (speed-test results)
│   └── syncMeta                           (installation credentials, last sync)
├── src/lib/api.ts                         ← Fetch wrapper + anonymous credentials
├── src/lib/sync.ts                        ← Sync orchestration
├── src/hooks/useSync.ts                   ← Sync status, auto-sync, manual sync
└── UI Pages
    ├── Dashboard (overview / offline / sync / activity)
    ├── Offline Workspace
    ├── Sync Centre
    ├── Network Test
    ├── Network Reports
    └── Settings
```

Optional backend connection:

```
Frontend ── JSON over HTTP ──▶ Node/Express API ──▶ PostgreSQL
         /api/v1/sync
         /api/v1/reports
         /api/v1/network-tests
         /api/v1/stats
```

---

6. Data Types (What Gets Stored)

Type	Fields	Stored Where
Workspace Record	id, title, body, category, pinned, timestamps	IndexedDB + optional backend
Pending Change	recordId, action (create/update/delete), changedAt	IndexedDB sync queue
Network Report	id, provider, location, issue, reportedAt, isDemo?, syncedAt?	IndexedDB + optional backend
Network Test	id, endpoint, latencyMs, downloadMbps, uploadMbps, testedAt, error, available, syncedAt?	IndexedDB + optional backend
Settings	dataSaver, compressImages, autoSync, etc.	IndexedDB
Installation Credentials	installationId, secret, createdAt	IndexedDB (never sent by server)

---

7. Mobile Providers Supported

- MTN
- Airtel
- Glo
- 9mobile
- Other (custom)

---

8. How to Run

Frontend (this folder)

```
cd /home/dazumi/ConnectEase
npm install
npm run dev          # http://localhost:5173
npm run build        # production build
CI=true npm run preview -- --port 4173
npx tsc --noEmit     # type-check
```

Backend (optional, separate repo)

```
cd /home/dazumi/ConnectEase_backend
npm install
cp .env.example .env   # set DATABASE_URL, INSTALLATION_PEPPER, CORS_ORIGINS
npm run migrate
npm run dev            # http://localhost:4000
curl http://localhost:4000/api/v1/health
```

Verification Already Performed

- Frontend `tsc --noEmit` — clean
- Frontend `npm run build` — passes
- Backend `tsc --noEmit` — clean
- Backend tests — 44 tests, 7 files, all passed
- Live end-to-end: workspace record → backend ✅
- Live end-to-end: network report → backend (`isDemo: false`) ✅
- Live end-to-end: network test → backend ✅
- Offline → online auto-sync ✅
- Remote delete via UI → backend 404 after delete ✅
- Mobile layout at 390px / 320px — no overflow ✅

- Service worker offline reload ✅

---

9. File Structure (ConnectEase Frontend)

```
src/
├── App.tsx                     # Routing, sync wiring, dashboard sections
├── main.tsx                    # Entry point
├── index.css                   # Tailwind styles
├── types.ts                    # All TypeScript interfaces
├── lib/
│   ├── utils.ts                # Date/format helpers, cn()
│   ├── db.ts                   # IndexedDB v3 + CRUD
│   ├── api.ts                  # API client + anonymous auth
│   ├── sync.ts                 # Sync orchestration
│   └── networkTest.ts          # Real latency/download/upload measurement
├── hooks/
│   ├── useHashRoute.ts         # Hash-based router
│   ├── useSync.ts              # Sync status & triggers
│   ├── useNetworkTests.ts      # Tests + reports state
│   ├── useWorkspaceData.ts     # Workspace records
│   └── ... (online status, service worker, settings)
├── components/
│   ├── Sidebar.tsx             # Main + dashboard-section navigation
│   ├── MobileNav.tsx           # Bottom mobile nav (6 destinations)
│   ├── Topbar.tsx              # Breadcrumbs + status
│   ├── DashboardSkeleton.tsx   # Loading state
│   ├── ConnectionCard.tsx      # Connection status + offline toggle
│   ├── DataSaverCard.tsx       # Data saver toggle
│   ├── StatCards.tsx           # Usage statistics
│   ├── OfflineLibrary.tsx      # Saved offline items
│   ├── SyncQueue.tsx           # Pending changes queue
│   ├── ActivityFeed.tsx        # Activity feed
│   ├── UsageCard.tsx           # Data usage card
│   ├── DemoNotice.tsx          # Explicit demo-data banner
│   ├── network-test/
│   │   ├── NetworkTestCard.tsx  # Test UI + history
│   │   └── NetworkTestPage.tsx  # Test page
│   └── network-report/
│       ├── ReportForm.tsx      # Report creation modal
│       ├── ReportList.tsx      # Search/filter/delete list
│       └── NetworkReportsPage.tsx
├── pages/
│   ├── SettingsPage.tsx
│   ├── SyncPage.tsx
│   └── WorkspacePage.tsx
├── data/
│   ├── demo.ts                 # Sample/demo data
│   └── navigation.ts           # Nav items, providers, labels
└── public/
    └── sw.js                   # Service worker (app shell cache)
```

---

10. Presenting This App

Opening line:

> "ConnectEase is an offline-first mobile-network diagnostics and reporting tool built for Nigerian users. It lets people measure real network performance, report poor connectivity by provider and location, and keep local records — all without needing an account or even an internet connection."

Key demo points:

1. Open the app — it works immediately (offline).
2. Dashboard shows demo data clearly marked as sample.
3. Go to Offline Workspace, create a record, refresh the page — it persists.
4. Go to Network Test, enter a URL, run a real measurement (no fake speeds).
5. Go to Network Reports, create a report with provider/location/issue, save it.
6. Click "Add sample reports" — see them clearly labelled "Demo data — not a real report."
7. Go to Sync Centre — see connection status; if a backend is configured, sync sends local data to the server.

Key message:

> "ConnectEase is honest about what is measured, honest about what is demo, and keeps the user's data on their device first — sync is a choice, not a requirement."

---

11. Related Files in This Repository

- `README.md` — Quick start & run instructions
- `integration.md` — Complete frontend ↔ backend contract, endpoints, sync sequence, troubleshooting
- `.env.example` — Frontend API configuration (`VITE_API_URL`)
- `package.json` — Scripts: `dev`, `build`, `preview`

Backend repository: `ConnectEase_backend` (separate folder/repo) — Express 5 + TypeScript + PostgreSQL with migrations, Zod validation, rate limiting, CORS, and 44 passing tests.

Prepared for presentation. All features described are implemented and verified in the current source code.